Hadoop Linux Setup

You

Original custom instructions no longer available

You 5/5/2026, 7:37:50 PM

write command for confgruing hadoop on local change system files

starting service

installing if not

simply having simple file system like given by ls command

```
Directory: E:\DSBDA-main\Hadoop_Word_Count
```

Mode LastWriteTime Length Name

---

d----- 4/21/2026 12:05 PM input
d----- 4/21/2026 12:05 PM output
d----- 4/21/2026 12:05 PM wordcount_classes
-a---- 4/21/2026 12:05 PM 187738 output11.png
-a---- 4/21/2026 12:05 PM 3345 wordcount.jar
-a---- 4/21/2026 12:05 PM 2816 WordCount.java

here input is input.txt in input folder and output is saved in output folder

i know it should be deleted so if existed delete it , i have pre jar files but give commands for creating it

also dont be childish give me only useful response.

also this file sturcture is for windows but give me complete confriguration for linux only because it is my work machine.

ChatGPT 5/5/2026, 7:37:51 PM

Below is a **clean, Linux-only Hadoop local (pseudo-distributed) setup + execution flow** tailored to your structure and requirements.

---

1) Install prerequisites (if not installed)

```
# Java (required)
sudo apt update
sudo apt install openjdk-8-jdk -y

# Verify
java -version

# SSH (required for Hadoop daemons)
sudo apt install openssh-server -y

# Enable SSH
ssh-keygen -t rsa -P ""
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
ssh localhost
```

---

2) Download & extract Hadoop

```
cd /opt
sudo wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
sudo tar -xvzf hadoop-3.3.6.tar.gz
sudo mv hadoop-3.3.6 hadoop
sudo chown -R $USER:$USER /opt/hadoop
```

---

3) Set environment variables

```
nano ~/.bashrc
```

Add:

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_HOME=/opt/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

Apply:

```
source ~/.bashrc
```

---

4) Configure Hadoop files

Go to config:

```
cd $HADOOP_HOME/etc/hadoop
```

---

core-site.xml

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

---

hdfs-site.xml

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>

  <property>
    <name>dfs.namenode.name.dir</name>
```

```
    <value>file:///home/$USER/hadoopdata/hdfs/namenode</value>
  </property>

  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///home/$USER/hadoopdata/hdfs/datanode</value>
  </property>
</configuration>
```

---

mapred-site.xml

```
cp mapred-site.xml.template mapred-site.xml
nano mapred-site.xml
```

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

---

yarn-site.xml

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

Set JAVA_HOME in Hadoop

```
nano hadoop-env.sh
```

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
```

---

5) Format HDFS (only once)

```
hdfs namenode -format
```

---

6) Start Hadoop services

```
start-dfs.sh
start-yarn.sh
```

Check:

```
jps
```

Expected:

```
NameNode
DataNode
ResourceManager
NodeManager
```

---

7) HDFS basic commands (like ls)

```
hdfs dfs -mkdir /input
hdfs dfs -ls /
```

---

8) Prepare your project structure (Linux)

Assume:

```
~/Hadoop_Word_Count/
  input/
    input.txt
  output/
  WordCount.java
```

---

9) Create JAR (even if you already have one)

```
cd ~/Hadoop_Word_Count

mkdir -p wordcount_classes

# Compile
javac -classpath `hadoop classpath` -d wordcount_classes WordCount.java

# Create jar
jar -cvf wordcount.jar -C wordcount_classes/ .
```

---

10) Upload input to HDFS

```
hdfs dfs -rm -r /output 2>/dev/null

hdfs dfs -mkdir -p /input
hdfs dfs -put input/input.txt /input/
```

---

11) Run WordCount

```
hadoop jar wordcount.jar WordCount /input /output
```

---

12) View output

```
hdfs dfs -ls /output
hdfs dfs -cat /output/part-r-00000
```

---

13) Clean output (mandatory before rerun)

```
hdfs dfs -rm -r /output
```

---

14) Stop services

```
stop-yarn.sh
stop-dfs.sh
```

Key Notes (no nonsense)

- Output directory **must not exist** → always delete before run
- Use HDFS paths (/input, /output), not local paths
- JAR must contain compiled .class, not .java
- hadoop classpath is required for compilation
- Run jps if anything fails

If you want, I can give a **minimal working WordCount.java (clean + correct imports)** so you don't hit runtime errors.